

Unit 2 — Text Preprocessing & Cleaning

Why Preprocessing Exists

Raw text is noisy: inconsistent casing, punctuation, typos, HTML tags, emojis, slang. Models (classical ML especially) perform far better on clean, normalized input. **Rule of thumb:** preprocessing choices depend entirely on your downstream task — this unit's biggest practical skill is knowing *which* steps to apply and which to skip.

1. The Standard Pipeline (Step by Step)

```
Raw text
→ Lowercasing
→ Noise removal (HTML, URLs, special chars)
→ Tokenization
→ Stopword removal
→ Stemming / Lemmatization
→ (Optional) POS tagging, spell correction
→ Clean tokens ready for vectorization
```

Step 1: Lowercasing

"Apple is GREAT" → "apple is great"

Why: reduces vocabulary size (Apple/apple/APPLE treated same).

When to skip: Named Entity Recognition (case signals proper nouns — "Apple" the company vs "apple" the fruit), or when using cased Transformer models (BERT-cased).

Step 2: Noise Removal

- Remove HTML tags: ``<p>Hello</p>`` → ``Hello`` (using regex or BeautifulSoup)

- Remove URLs: "Check `http://x.com` now" → "Check now"
- Remove/expand contractions: "don't" → "do not"
- Remove extra whitespace, non-ASCII characters, digits (task-dependent)
- Handle emojis: convert to text (😊 → "smile") or remove, depending on if sentiment matters

Step 3: Tokenization

Splitting text into units (words, subwords, or sentences).

- **Word tokenization:** "I love NLP!" → ["I", "love", "NLP", "!"]
- **Sentence tokenization:** splits paragraph into sentences (careful with abbreviations: "Dr. Smith went to the U.S." shouldn't split at "Dr." or "U.S.")

- **Subword tokenization** (used by modern LLMs — covered deeply in Unit 7): Byte-Pair Encoding (BPE), WordPiece. "unhappiness" → ["un", "happiness"] or ["un", "happi", "ness"]. This solves the out-of-vocabulary (OOV) problem — rare words are broken into known pieces.

Common doubt: *Why not just split on spaces?*

Because of punctuation ("NLP!" ≠ "NLP"), contractions ("don't" is 1 or 2 tokens?), and languages without spaces (Chinese, Japanese) — proper tokenizers handle these edge cases with rules or learned models.

Step 4: Stopword Removal

Remove high-frequency, low-information words ("the", "is", "at").

```
from nltk.corpus import stopwords
tokens = ["this", "movie", "is", "not", "good"]
filtered = [w for w in tokens if w not in stopwords.words('english')]
# Result: ["movie", "good"]  △ Note: "not" got removed — sentiment flipped incorrect
```

Practical warning: Always check if your stopword list removes negation words like "not", "no", "never" — this can destroy sentiment meaning. Customize the stopword list per task.

When to use: BoW/TF-IDF classical pipelines, topic modeling, search indexing.

When to skip: Machine translation, question answering, any Transformer-based model (they need full grammatical structure).

Step 5: Stemming vs Lemmatization (Step by Step)

Stemming — rule-based suffix stripping (e.g., Porter Stemmer algorithm):

```
"studies" → "studi"
"studying" → "studi"
"caring" → "care" → sometimes over-stems: "carin"
```

Fast but crude — output may not be a real word.

Lemmatization — uses a dictionary + POS tag to find the real base form:

```
"studies" (verb) → "study"
"better" (adjective) → "good"
"was" (verb) → "be"
```

Slower but linguistically accurate.

Decision rule:

Situation	Choose
Need speed, large-scale search indexing	Stemming

Situation	Choose
Need accuracy, sentiment/legal/medical text	Lemmatization
Using deep learning/Transformers	Neither — use subword tokenization instead

Step 6: Spell Correction (Optional, task-dependent)

Useful for social media/user-generated text with typos: "recieve" → "receive". Tools: SymSpell, TextBlob, or edit-distance based correction. Skip for formal/clean corpora (news, legal documents) — unnecessary compute cost.

2. Full Worked Example

Input: "I can't believe how AMAZING this restaurant was!! Visit: <http://foo.com> 😊"

Step	Output
Lowercase	i can't believe how amazing this restaurant was!! visit: http://foo.com 😊
Expand contractions	i can not believe how amazing this restaurant was!! visit: http://foo.com 😊
Remove URL	i can not believe how amazing this restaurant was!! visit:
Remove emoji/punctuation	i can not believe how amazing this restaurant was visit
Tokenize	[i, can, not, believe, how, amazing, this, restaurant, was, visit]
Remove stopwords (keep "not")	[not, believe, amazing, restaurant, visit]
Lemmatize	[not, believe, amazing, restaurant, visit]

This clean token list now feeds into Unit 3 (vectorization).

3. Code Implementation (Practical, Python/NLTK & spaCy)

```
import re, nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

text = "I can't believe how AMAZING this restaurant was!! Visit: http://foo.com"

text = text.lower()
text = re.sub(r"http\S+", "", text)          # remove URLs
text = re.sub(r"can't", "can not", text)      # basic contraction expansion
text = re.sub(r"^[a-z\s]", "", text)         # remove punctuation/digits

tokens = word_tokenize(text)
stop_words = set(stopwords.words('english')) - {"not", "no", "never"}
tokens = [t for t in tokens if t not in stop_words]

lemmatizer = WordNetLemmatizer()
tokens = [lemmatizer.lemmatize(t) for t in tokens]
```

```
print(tokens)
# ['believe', 'amazing', 'restaurant', 'visit']
```

For production-grade pipelines, **spaCy** is preferred over NLTK — faster, has built-in tokenization + lemmatization + POS in one pass:

```
import spacy
nlp = spacy.load("en_core_web_sm")
doc = nlp("I can't believe how amazing this restaurant was!!!")
tokens = [tok.lemma_ for tok in doc if not tok.is_stop and not tok.is_punct]
```

4. Which Tool/Approach to Choose

Need	Use
Quick prototyping, teaching, simple projects	NLTK
Production pipelines, speed, POS/NER combined	spaCy
Subword tokenization for deep learning	Hugging Face tokenizers (BPE/WordPiece)
Regex-only lightweight cleaning	Python re module

5. Common Doubts (Theory + Practical)

Q: Should I always remove stopwords?

No — for BoW/TF-IDF + classical ML: usually yes. For Transformers/LLMs: no, they use full context including stopwords via attention.

Q: Does preprocessing hurt deep learning models?

Yes, often. Heavy preprocessing (removing stopwords, punctuation, aggressive stemming) throws away information that neural models — especially Transformers — can actually use. Modern practice: minimal cleaning (fix encoding, remove HTML) + subword tokenization, let the model learn the rest.

Q: Why do case and negation errors happen so often in real projects?

Because default stopwords lists and lowercase are applied blindly. Always inspect a sample of your cleaned output before training — a very common real-world bug.

Q: How does this map to real projects?

- Spam filters: aggressive cleaning + stemming + BoW (Unit 4).
- Chatbots/LLM apps: minimal cleaning, rely on subword tokenizer (Unit 7).
- Search engines: lemmatization + stopwords removal for indexing efficiency.